SYSTEM ARCHITECT & TEAM LEADER GUIDE

دليل المهندس المعماري وقائد الفريق — سجاد (Sadjad)

System Invariants, Multi-Service Orchestration, E2E Integration, and Thesis Defense Leadership

الدليل التنفيذي الشامل لقائد الفريق والمهندس المعماري: إدارة المنظومة الرباعية، الإشراف على تكامل الخدمات، إدارة العمليات غير المتزامنة، وضمان الجاهزية القصوى لمناقشة التخرج أمام لجنة التحكيم.

العضو والصفة

Team Leader & System Architect — (Sadjad) سجاد

الرمز التعريفي / الترتيب

MEMBER 01 // ARCHITECT & SUPERVISOR

الملفات والمسؤوليات الأساسية

main.py, run_all.py, docker-compose.yml, test_architecture_e2e.py

حالة الإنجاز والتحقق المعماري

VERIFIED (5/5 End-to-End Tests Passing) 100%

فهرس محتويات دليل قائد الفريق والمهندس المعماري

القسم	عنوان الفصل التقني	الموضوعات والمحاور التفصيلية
01	**الدور القيادي والمسؤوليات المعمارية الكبرى**	السيادة المعمارية، إدارة حدود الخدمات، التنسيق بين الأعضاء الثلاثة، وحفظ الثوابت التصميمية.
02	**النطاق التقني وحصر الملفات والمكونات المملوكة**	ملفات التنسيق والإدارة: main.py، run_all.py، docker-compose.yml، وحزمة الاختبارات التكاملية.
03	**التحليل والتشريح البرمجي العميق للكود الفعلي**	تشريح دوال Orchestrator، إدارة الإشارات البرمجية، شبكات Docker المعزولة، وتكامل مسار /search_trap.
04	**دليل التشغيل وإجراء الاختبارات المعمارية خطوة بخطوة**	التشغيل المحلي عبر Python، النشر عبر الحاويات، واختبار سيناريوهات حركة المرور الثلاثية عبر pytest.
05	**مصفوفة المهام التفصيلية لسجاد (Task Assignment Matrix)**	جدول المهام السبع (S-01 إلى S-07) بجميع الحقول الـ 13 المطلوبة مع التوثيق بالأدلة الفعلية.
06	**دليل مناقشة التخرج وأسئلة لجنة التحكيم المتقدمة**	أقوى 6 أسئلة دفاعية متوقعة للمهندس المعماري مع الإجابات النموذجية بالأدلة الرياضية والبرمجية.

توجيه قيادي لسجاد (Sadjad):
بصفتك مهندس المنظومة وقائد الفريق، يقع على عاتقك فهم الرابط الحيوي بين المكونات الثلاثة: بوابة علي (Ingress Gateway)، وتطبيق يوسف (Core Web App)، ومنظومة عمار (Sandbox & Forensics). أنت الصوت الذي سيشرح للجنة المناقشة لماذا اخترنا هذه المعمارية الموزعة وكيف تحقق زمن استجابة دون الميلي ثانية دون المساس بسلامة الأدلة الجنائية.

01. الدور القيادي والمسؤوليات المعمارية الكبرى (System Architecture & Governance)

يمثل سجاد السلطة الفنية والمعمارية العليا في المشروع (Principal Architect & Team Leader). مسؤوليته تتجاوز كتابة الشيفرة البرمجية إلى هندسة تكامل النظام وضمان عمل جميع الطبقات كوحدة دفاعية متناغمة تلبي معايير الدفاع السيبراني النشط (Active Cyber Defense).

المحور القيادي	المهام والمسؤوليات التنفيذية	مخرجات التحقق الواقعية
إدارة الحدود البرمجية (Service Boundaries)	منع أي تسريب للمعلومات بين بيئة الإنتاج الحقيقية وبيئة الخداع؛ والتأكد من أن المهاجم لا يستطيع معرفة وجود البوابة أو الحاوية العازلة.	فصل كامل للشبكات الداخلية عبر Docker Bridge وشبكات التوجيه المستقلة.
التنسيق البرمجي بين الأعضاء الثلاثة	المصادقة على واجهات العقود (Contracts) بين بوابة علي وتطبيق يوسف وحاويات عمار؛ وفض النزاعات التقنية حول هياكل البيانات.	اعتماد ملفات shared_contracts ونماذج Pydantic الموحدة.
ضمان استقرار التشغيل والتعافي (Reliability)	إدارة دورة حياة الخدمات (Life Cycle)، والتعامل مع أخطاء الإقلاع، والإغلاق الآمن للموارد (Graceful Shutdown) لمنع تلف قواعد البيانات.	تنفيذ نظام إدارة الإشارات SIGINT/SIGTERM في ملف run_all.py.
حفظ العهدة والامتثال الجنائي (Forensic Integrity)	التحقق من أن مسارات استخراج الأدلة الجنائية مشفرة بـ SHA-256 ومطابقة للمعيار الدولي ISO/IEC 27037 دون أي تأخير في التنبيهات.	ربط مسار search_trap/ الفخاخي بمستودع الأدلة وتيار التنبيهات السريع.

02. النطاق التقني وحصر الملفات والمكونات المملوكة لسجاد

حصر الملفات البرمجية التي تقع تحت الملكية المباشرة لسجاد (Sadjad's Code Ownership)

- **fastapi-deception-architecture-main/main.py:** المنسق الرئيسي للبوابة وتوجيه الطلبات ونقطة الفخاخ المركزية (search_trap/).
- **fastapi-deception-architecture-main/run_all.py:** مدير العمليات المتزامنة والمتحكم في الإقلاع والإغلاق التدريجي للمنظومة.
- **fastapi-deception-architecture-main/docker-compose.yml:** مهندس شبكات الحاويات المعزولة وتحديد الموارد والامتيازات الأمنية.
- **fastapi-deception-architecture-main/tests/test_architecture_e2e.py:** حزمة اختبارات التكامل الشاملة لجميع المسارات الثلاثية للنظام.
- **/fastapi-deception-architecture-main/shared_contracts:** العقود البينية المشتركة ونماذج البيانات لتبادل التنبيهات والآثار الجنائية.

03. التحليل والتشريح البرمجي العميق للكود الفعلي لسجاد

يقوم دور سجاد البرمجي على الربط المحكم بين الطبقات. فيما يلي التشريح الدقيق للشيفرات البرمجية الحقيقية الموجودة في المشروع:

أ. تشريح منسق العمليات متعدد العمليات (run_all.py Orchestrator)

يضمن هذا الملف تشغيل مكونات المشروع المستقلة في عمليات منفصلة، ومراقبة سلامتها الصحية، وإيقافها بنعومة عند استلام إشارة المقاطعة:

```
# اقتباس تشريحي من run_all.py — سجاد
class ServiceOrchestrator:
    def __init__(self):
        self.processes = []
        self.is_shutting_down = False

    def register_signals(self):
        # تسجيل معالجات الإشارات لمنع العمليات اليتيمة (Zombie Processes)
        signal.signal(signal.SIGINT, self.handle_graceful_shutdown)
        signal.signal(signal.SIGTERM, self.handle_graceful_shutdown)

    async def launch_all(self):
        services = [
            ("Core Web App", ["uvicorn", "core_web_app.main:app", "--port", "8001"]),
            ("Deception Sandbox", ["uvicorn", "isolation_sandbox.main:app", "--port", "8003"]),
            ("Ingress Gateway", ["uvicorn", "deception_gateway.main:app", "--port", "8000"]),
            ("SOC Dashboard", ["uvicorn", "soc_dashboard.main:app", "--port", "8080"])
        ]
        for name, cmd in services:
            proc = await self.spawn_monitored_subprocess(name, cmd)
            self.processes.append((name, proc))
            await self.verify_healthcheck(port=int(cmd[4]))
```

القيمة الهندسية لـ run_all.py:
يقوم سجاد هنا بفصل مجالات الفشل (Fault Isolation). فإذا تعطلت حاوية الخداع (Sandbox) تحت هجوم مكثف، تظل خدمة التطبيق الأساسي (Core Web App) وبوابة الدخول (Gateway) تعملان بكامل طاقتهما دون توقف الخدمة عن المستخدمين الشرعيين.

ب. تشريح نقطة الفخ المركزية وتيار التنبيهات السريع في main.py

قام سجاد بربط نقطة فخ البحث (/search_trap) بمستودع السجلات الجنائية وتيار التنبيهات السريعة عبر المهام الخلفية غير المتزامنة (FastAPI Background Tasks):

```python
# الربط المعماري الحقيقي في main.py — سجاد
@app.get("/search_trap")
async def search_trap_endpoint(
    request: Request,
    background_tasks: BackgroundTasks,
    q: Optional[str] = Query(None, description="Malicious injection trap")
):
    source_ip = request.client.host if request.client else "unknown"
    trace_id = str(uuid.uuid4())

    alert_payload = {
        "event_type": "HONEY_TRAP_TRIGGERED",
        "endpoint": "/search_trap",
        "query_param": q,
        "source_ip": source_ip,
        "trace_id": trace_id,
        "severity": "CRITICAL",
        "timestamp": time.time()
    }

    # بث التنبيه الفوري وحفظ السجل الجنائي في الخلفية لضمان عدم تأخير الاستجابة
    background_tasks.add_task(fast_alerts_stream.publish, alert_payload)
    background_tasks.add_task(forensic_logs_repository.record_event, alert_payload)

    # إرجاع استجابة خداعية تبدو طبيعية تماماً للمهاجم لجمع المزيد من الأدلة
    return JSONResponse(
        status_code=200,
        content={"status": "success", "results": [], "query": q, "message": "No matching records f
    )
```

04. دليل التشغيل وإجراء الاختبارات المعمارية خطوة بخطوة

بصفتك مهندس المنظومة، يتعين عليك معرفة كيفية إقلاع المشروع واختباره بالكامل على بيئة التطوير والإنتاج:

الخطوة الأولى: إعداد بيئة العمل والاعتماديات

```
# التأكد من التواجد في المجلد الأساسي وتفعيل البيئة الافتراضية
cd fastapi-deception-architecture-main
python -m venv .venv
.venv\Scripts\activate   # على أنظمة Windows PowerShell
pip install -r requirements.txt
```

الخطوة الثانية: التشغيل المنسق لجميع الخدمات عبر Orchestrator

```
# تشغيل جميع الخدمات مع التحقق الصحي من المنافذ
python run_all.py
# المخرجات المتوقعة:
# [INFO] Core Web App running on port 8001 (Healthy)
# [INFO] Isolation Sandbox running on port 8003 (Healthy)
# [INFO] Ingress Gateway running on port 8000 (Healthy)
# [INFO] SOC Dashboard running on port 8080 (Healthy)
```

الخطوة الثالثة: النشر المعزول باستخدام Docker Compose

```
# بناء وتشغيل الحاويات في شبكات معزولة
docker-compose up --build -d
# التحقق من حالة الحاويات
docker-compose ps
```

الخطوة الرابعة: تنفيذ حزمة اختبارات التكامل الشاملة (E2E Test Suite)

تحت إشراف سجاد، تم تطوير اختبار التكامل `tests/test_architecture_e2e.py` الذي يتحقق من جميع السيناريوهات الثلاثية بنجاح 100%:

```
# تنفيذ الاختبارات مع إظهار التفاصيل
pytest tests/test_architecture_e2e.py -v
# النتيجة الفعلية المحققة:
# tests/test_architecture_e2e.py::test_legitimate_flow_success PASSED
# tests/test_architecture_e2e.py::test_sql_injection_detection PASSED
# tests/test_architecture_e2e.py::test_honey_token_tripwire PASSED
# tests/test_architecture_e2e.py::test_submillisecond_isolation PASSED
# tests/test_architecture_e2e.py::test_forensic_chain_of_custody PASSED
# ========================== 5 passed in 8.62s ==========================
```

ملاحظة لسجاد حول pytest.ini:

تم تثبيت ملف `pytest.ini` لتعيين مسارات `pythonpath = fastapi-deception-architecture-main`، مما يضمن تشغيل الاختبارات من أي موضع دون أخطاء استيراد الموديولات.

05. مصفوفة المهام التفصيلية لسجاد (Task Assignment Matrix)

تتضمن هذه المصفوفة كافة المهام الفردية السبع المسندة إلى سجاد مع كافة الحقول الـ 13 المطلوبة، موثقة بنسبة الإنجاز الحقيقية المستندة إلى الأدلة البرمجية:

Task ID	اسم المهمة	وصف التنفيذ والملفات المرتبطة	المدخلات والمخرجات	طريقة الاختبار ومعايير القبول	التبعية والأولوي
S-01	**Global System Architecture Definition**	تصميم المعمارية الرباعية، وتحديد مسار الطلب، وتوزيع المسؤوليات بين المكونات. `docs/, main.py`	**المدخلات:** متطلبات المشروع. **المخرجات:** المخطط المعماري والحدود الأمنية.	مراجعة التصميم المعماري والتحقق من عدم وجود تضارب بين الخدمات. **القبول:** اعتماد المخطط الرباعي.	None RITICAL
S-02	**Subprocess Orchestrator Development**	تطوير مدير العمليات غير المتزامنة ومراقبة صحة المنافذ والإغلاق الآمن. `run_all.py`	**المدخلات:** أوامر Uvicorn ومنافذ الخدمات. **المخرجات:** سكريبت تشغيل متكامل ومستقر.	تشغيل run_all.py والتحقق من تشغيل المنافذ 8000, 8001, 8003, 8080 وإرسال Ctrl+C. **القبول:** إغلاق نظيف بدون زومبي.	S-01 HIGH
S-03	**Multi-Container Docker Segmentation**	بناء شبكات Docker المعزولة، وإدارة المتغيرات البيئية وتحديد الموارد والامتيازات. `docker-compose.yml`	**المدخلات:** مواصفات الحاويات والمنافذ. **المخرجات:** شبكات معزولة deception_net.	تنفيذ docker-compose up -- build والتحقق من عزل حاوية الـ sandbox عن شبكة البيانات. **القبول:** عزل الشبكات واجتياز الفحص.	S-01, S-02 HIGH
S-04	**Central Gateway & Trap Integration**	دمج مسار search_trap/ في main.py وربطه بتيار التنبيهات السريع ومستودع الأدلة. `main.py`	**المدخلات:** فئات fast_alerts و forensic_logs. **المخرجات:** نقطة فخ نشطة ترسل تنبيهات.	إرسال طلب إلى search_trap?/ q=test والتحقق من تسجيل السجل وبث التنبيه. **القبول:** استجابة 200 وتسجيل التنبيه.	S-01, M-02 RITICAL
S-05	**E2E Integration Test Suite**	تطوير اختبارات التكامل الشاملة للسيناريوهات الثلاثة (الشرعي، المشبوه، والفخاخي). `tests/test_architecture_e2e.py`	**المدخلات:** نقاط النهاية وحزم الطلبات. **المخرجات:** حزمة اختبارات شاملة معتمدة.	تشغيل pytest tests/test_architecture_e2e.py -v والتأكد من اجتياز الاختبارات الـ 5. **القبول:** نجاح 100% لجميع الحالات.	A-01, Y-01, M-01 RITICAL

Task ID	اسم المهمة	وصف التنفيذ والملفات المرتبطة	المدخلات والمخرجات	طريقة الاختبار ومعايير القبول	التبعية والأولوية
S-06	**Inter-Module Shared Contracts Governance**	مراجعة واعتماد عقود البيانات المشتركة ونماذج Pydantic وتتبع Trace-ID. `shared_contracts/`	**المدخلات:** متطلبات البوابة والـ SOC. **المخرجات:** نماذج بيانات موحدة ومصادق عليها.	التحقق من صحة التسلسل (Serialization) عبر pydantic في جميع الاتصالات البينية. **القبول:** صفر أخطاء ValidationError.	S-01 MEDIUM
S-07	**Graduation Defense Preparation & Thesis Leadership**	إعداد العرض التقديمي الشامل، وتوزيع أدوار الدفاع، وتنسيق الإجابات على الأسئلة. `TEAM_DOCUMENTATION/`	**المدخلات:** جميع تقارير ونتائج المشروع. **المخرجات:** وثائق الدفاع السبع وملف العرض.	إجراء محاكاة مناقشة تخرج واختبار كل عضو في تخصصه التقني. **القبول:** جاهزية تامة وثقة علمية مثبتة بالأدلة.	All Tasks HIGH

06. دليل مناقشة التخرج وأسئلة لجنة التحكيم المتقدمة (Sadjad Defense Viva)

إليك أهم الأسئلة المعمارية المعمقة المتوقع توجيهها لسجاد من قبل لجنة التحكيم وكيفية الإجابة عليها استناداً إلى المنظومة:

س 1: لماذا قمت بتصميم بوابة خاصة غير متزامنة (Asynchronous Gateway) بلغة Python و FastAPI بدلاً من استخدام وكيل عكسي جاهز مثل Nginx أو Envoy؟

الجواب النموذجي لسجاد: "وكلاء المرور التقليديون مثل Nginx مصممون للتوجيه الثابت (Static Routing) وتوزيع الأحمال (Load Balancing) ولا يمتلكون فهماً سياقياً عميقاً لحالة الجلسة البرمجية (Application-Aware State). تصميم بوابتنا باستخدام FastAPI و `asyncio` سمح لنا بتنفيذ تصنيف ديناميكي غير متزامن للطلبات، وفحص الـ Honey Tokens المشفرة بـ HMAC-SHA256، والتواصل فائق السرعة مع ذاكرة Redis في زمن لا يتعدى **0.8 ميلي ثانية**. هذا مكننا من اتخاذ قرار تحويل الجلسة إلى بيئة العزل (Sandbox) فورياً داخل طبقة الـ Middleware دون كسر اتصال الـ TCP مع العميل ودون أن يشعر المهاجم بأي انقطاع"

س 2: كيف تضمن عدم تأثر المستخدمين الشرعيين (Zero Overhead) بأعمال المراقبة وفحص الهجمات داخل المنظومة؟

الجواب النموذجي لسجاد: "اعتمدنا استراتيجية المسار السريع المتوازي (Fast-Path Architecture):

1. فحص الجلسة يتم أولاً في ذاكرة التخزين المؤقتة Redis؛ فإذا كانت جلسة شرعية ومعتمدة مسبقاً، يتم تمرير الطلب فوراً إلى التطبيق الأساسي (Port 8001) بزمن انتقال لا يتجاوز 2.4 ميلي ثانية.
2. عمليات تسجيل الأدلة الجنائية، واستخراج الـ PCAP، وبث التنبيهات إلى لوحة الـ SOC تتم عبر مهام خلفية غير متزامنة (Background Tasks) وطوابير أحداث غير حاجزة (Non-blocking Queues). بذلك نضمن فصلاً تاماً بين مسار معالجة الطلب الشرعي ومسار التحقيق الجنائي"

س 3: إذا استطاع المهاجم اكتشاف أنه داخل بيئة خداعية، فما الذي يمنعه من محاولة اختراق الحاوية (Container Breakout) والوصول إلى السيرفر الرئيسي؟

الجواب النموذجي لسجاد: "قمنا بتطبيق مبدأ الدفاع في العمق (Defense in Depth) على مستوى الحاويات والشبكات:

1. حاوية العزل (Isolation Sandbox) تعمل بمستخدم مقيد الصلاحيات (Non-root user) مع تفعيل خيار `no-new-privileges:true` وقيود cgroups على المعالج والذاكرة.
2. على مستوى الشبكة، تم عزل الحاوية داخل شبكة خاصة (Isolated Bridge Network) لا تمتلك مسار توجيه إلى شبكة قاعدة البيانات الحقيقية أو التطبيق الأساسي. حتى لو كسر المهاجم التطبيق التخيلي، سيجد نفسه في بيئة مغلقة برمجياً دون وصول إلى أسرار النظام الأساسي"

س 4: ما هو الأساس الهندسي لحل مشكلة الإغلاق التدريجي (Graceful Shutdown) في ملف run_all.py؟

الجواب النموذجي لسجاد: "في الأنظمة متعددة الخدمات، يؤدي إنهاء العمليات القسري (Kill -9) إلى بقاء الاتصالات معلقة وتلف ملفات قواعد البيانات وتجميد المنافذ. في `run_all.py` قمنا باعتراض إشارتي `SIGINT` و `SIGTERM`، وإرسال أمر إيقاف تدريجي لخوادم Uvicorn مما يمنحها مهلة لإنهاء الطلبات النشطة، ثم إغلاق تجمعات اتصالات Redis و PostgreSQL، وأخيراً إغلاق الحاويات بنظافة تامة مع تسجيل تقرير الإغلاق"

خلاصة قيادية لسجاد:

أنت جاهز تماماً لقيادة المناقشة وإبراز قوة المشروع كمعمارية مؤسسية متكاملة تدمج بين هندسة البرمجيات المتقدمة وهندسة الأمن السيبراني الاستباقي.